Final Project

Anya

2023-06-29

Data: https://www.kaggle.com/datasets/akshaydattatraykhare/diabetes-dataset (https://www.kaggle.com/datasets/akshaydattatraykhare/diabetes-dataset)

Research Question: Given a female's medical history, is it possible to accurately predict whether she was diagnosed with diabetes?

Diabetes is extremely prevalent in the world today, and the variables in the data set that seemed to have the most correlation with a positive diabetes diagnosis were the Glucose-level, Blood Pressure, and BMI. I wanted to verify if these three variables had an impact on the diagnosis and whether or not it is possible to predict whether someone will be diagnosed with diabetes solely from these variables. In order to test this, I first visualized the data set and these three variables with a wide variety of graphs to determine the variance each of these individual variables cover. I then created an SVM Model, Logistic Regression Model, and Decision Tree Model, all of which are well-versed in classification, and observed which of these three models performed the best.

Reading the Data and storing it into the variable Diabetes

```
Diabetes<-read.csv("/Users/anyalachwani/stanford/diabetes.csv",header=T,na.strings ="?")
```

Exploring the Data Set

```
fix(Diabetes)
dim(Diabetes)
```

```
## [1] 768   9
```

```
summary(Diabetes)
```

```
##   Pregnancies        Glucose      BloodPressure    SkinThickness
##  Min.   : 0.000   Min.   :  0.0   Min.   :  0.00   Min.   : 0.00
##  1st Qu.: 1.000   1st Qu.: 99.0   1st Qu.: 62.00   1st Qu.: 0.00
##  Median : 3.000   Median :117.0   Median : 72.00   Median :23.00
##  Mean   : 3.845   Mean   :120.9   Mean   : 69.11   Mean   :20.54
##  3rd Qu.: 6.000   3rd Qu.:140.2   3rd Qu.: 80.00   3rd Qu.:32.00
##  Max.   :17.000   Max.   :199.0   Max.   :122.00   Max.   :99.00
##     Insulin           BMI        DiabetesPedigreeFunction      Age
##  Min.   :  0.0   Min.   : 0.00   Min.   :0.0780           Min.   :21.00
##  1st Qu.:  0.0   1st Qu.:27.30   1st Qu.:0.2437           1st Qu.:24.00
##  Median : 30.5   Median :32.00   Median :0.3725           Median :29.00
##  Mean   : 79.8   Mean   :31.99   Mean   :0.4719           Mean   :33.24
##  3rd Qu.:127.2   3rd Qu.:36.60   3rd Qu.:0.6262           3rd Qu.:41.00
##  Max.   :846.0   Max.   :67.10   Max.   :2.4200           Max.   :81.00
##     Outcome
##  Min.   :0.000
##  1st Qu.:0.000
##  Median :0.000
##  Mean   :0.349
##  3rd Qu.:1.000
##  Max.   :1.000
```

```
names(Diabetes)
```

```
## [1] "Pregnancies"              "Glucose"
## [3] "BloodPressure"            "SkinThickness"
## [5] "Insulin"                  "BMI"
## [7] "DiabetesPedigreeFunction" "Age"
## [9] "Outcome"
```

```
head(Diabetes)
```

```
##   Pregnancies Glucose BloodPressure SkinThickness Insulin  BMI
## 1           6     148            72            35       0 33.6
## 2           1      85            66            29       0 26.6
## 3           8     183            64             0       0 23.3
## 4           1      89            66            23      94 28.1
## 5           0     137            40            35     168 43.1
## 6           5     116            74             0       0 25.6
##   DiabetesPedigreeFunction Age Outcome
## 1                    0.627  50       1
## 2                    0.351  31       0
## 3                    0.672  32       1
## 4                    0.167  21       0
## 5                    2.288  33       1
## 6                    0.201  30       0
```

There were no missing fields in the data I was going to use for my model. This can be seen because the sums of missing fields for the predictors I was using(Glucose, Blood Pressure, and BMI, were all 0)

```
dim(Diabetes[is.na(Diabetes$Glucose),])
```

```
## [1] 0 9
```

```
dim(Diabetes[is.na(Diabetes$BloodPressure),])
```

```
## [1] 0 9
```

```
dim(Diabetes[is.na(Diabetes$BMI),])
```

```
## [1] 0 9
```

```
sum(is.na(Diabetes$Glucose))
```

```
## [1] 0
```

```
sum(is.na(Diabetes$BloodPressure))
```

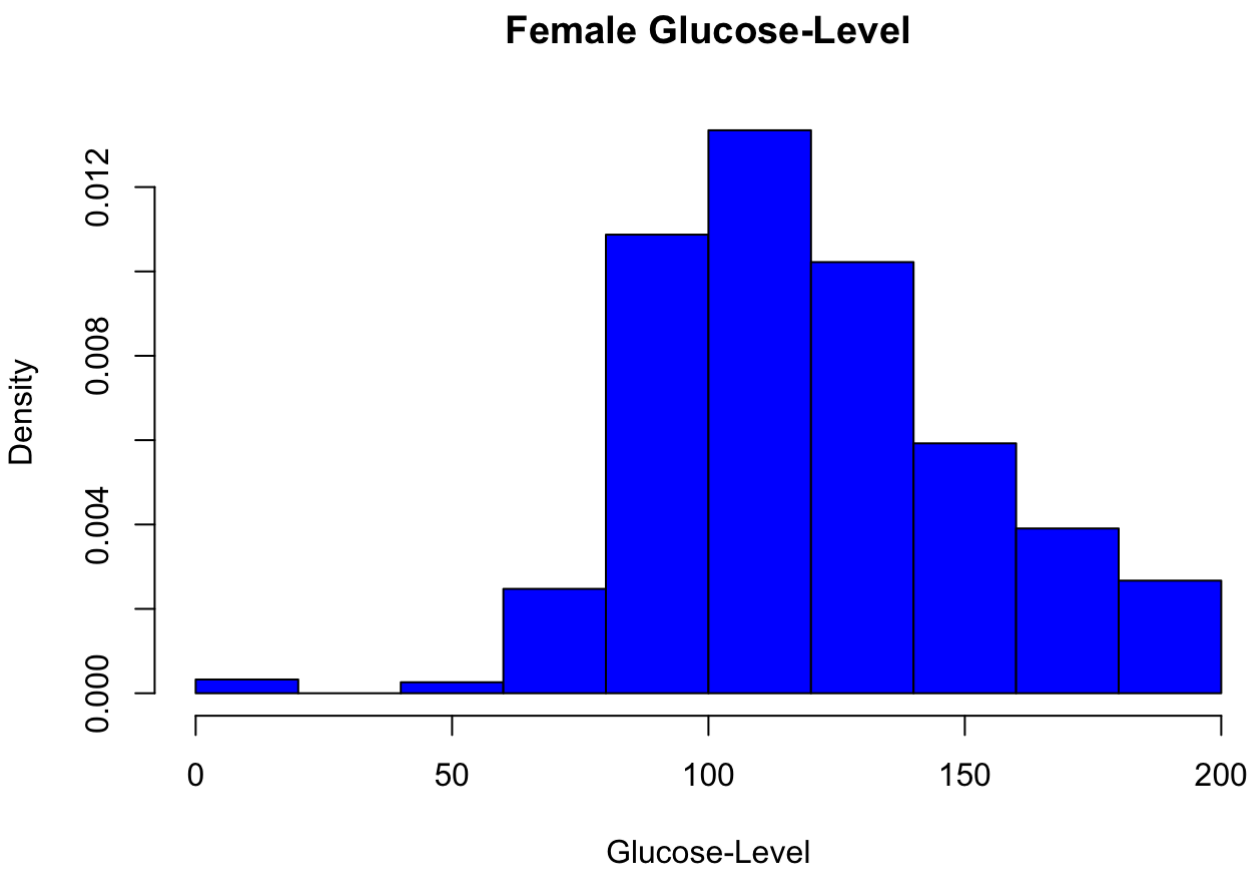
```
## [1] 0
```

```
sum(is.na(Diabetes$BMI))
```

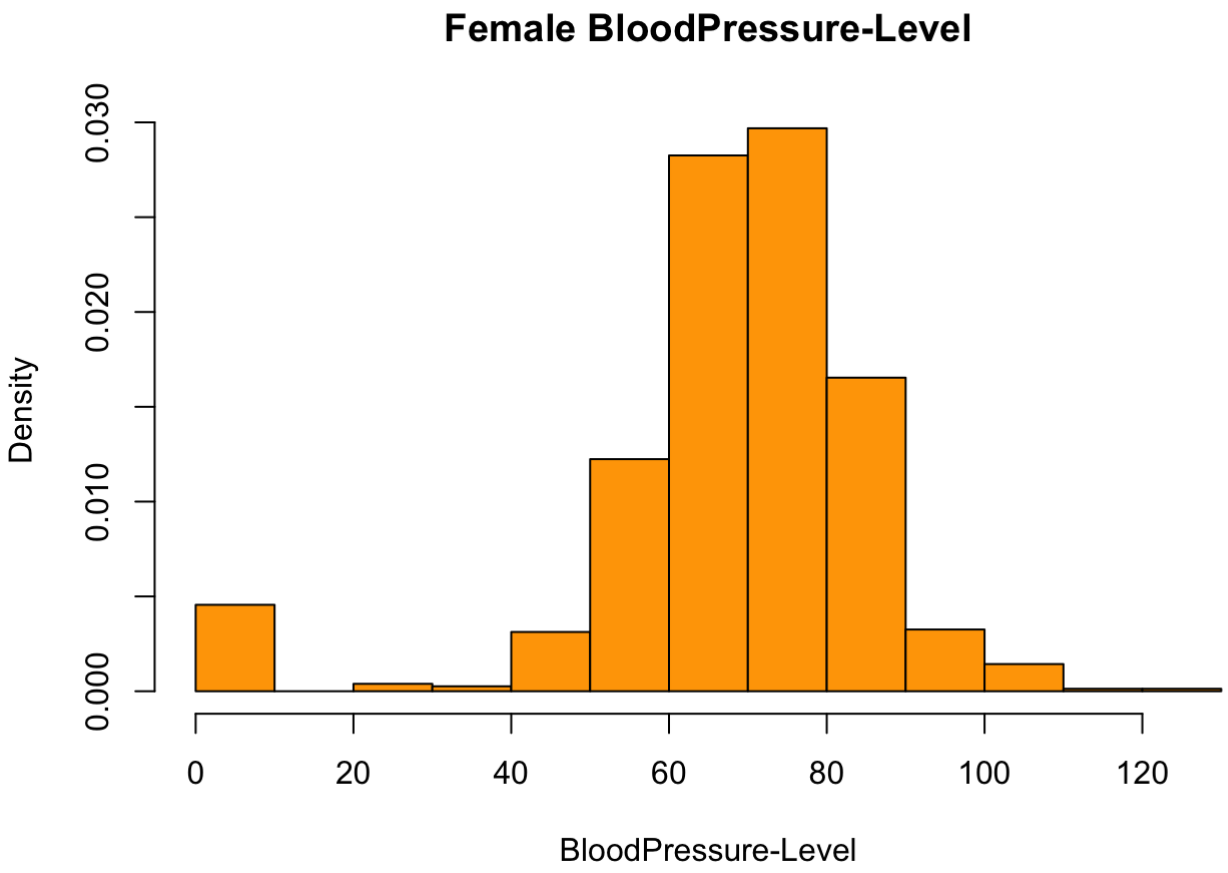
```
## [1] 0
```

Creating histograms for each of the predictors I am using.

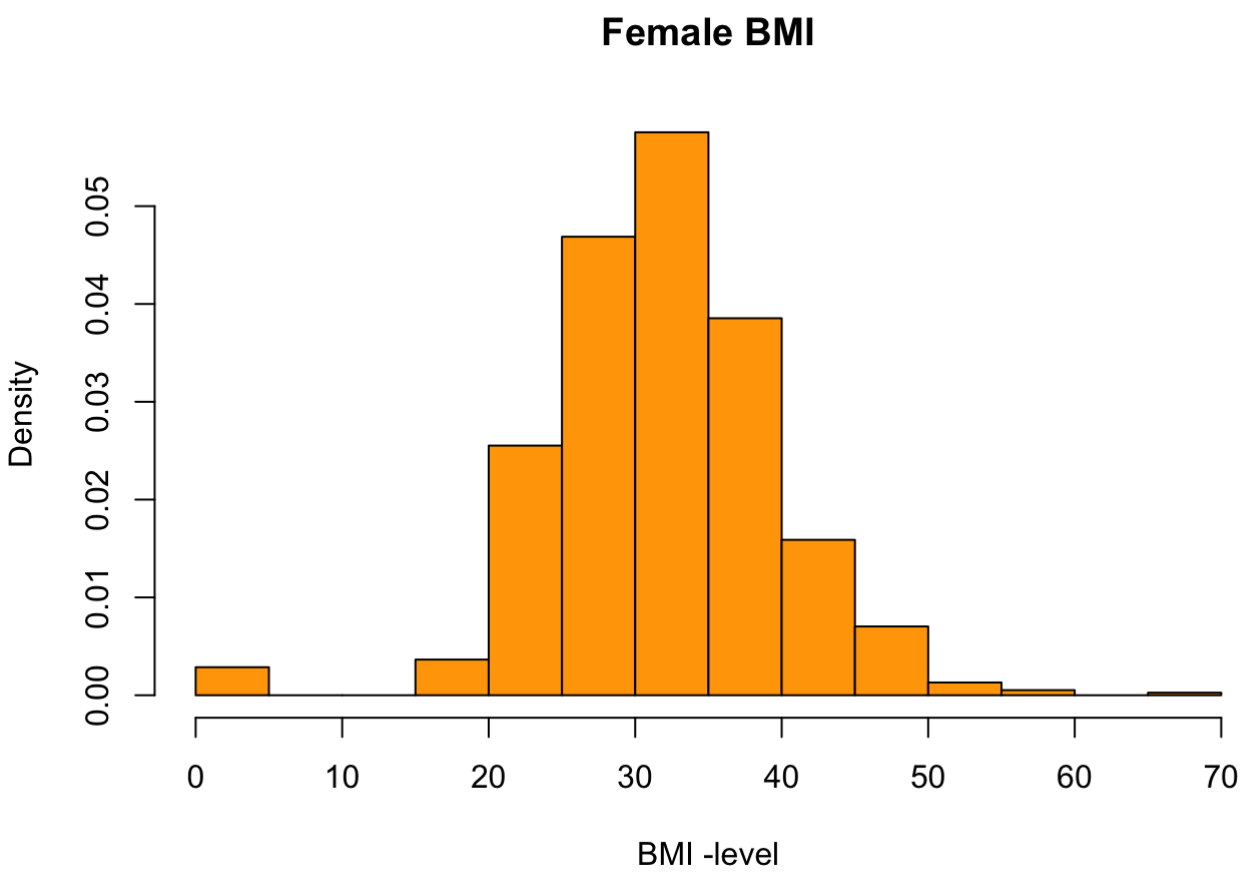
```
hist(Diabetes$Glucose,
main="Female Glucose-Level",
xlab="Glucose-Level",
col="blue",
freq=FALSE
)
```




```
hist(Diabetes$BloodPressure,
main="Female BloodPressure-Level",
xlab="BloodPressure-Level",
col="orange",
freq=FALSE
)
```




```
hist(Diabetes$BMI,
main="Female BMI",
xlab="BMI -level",
col="orange",
freq=FALSE
)
```

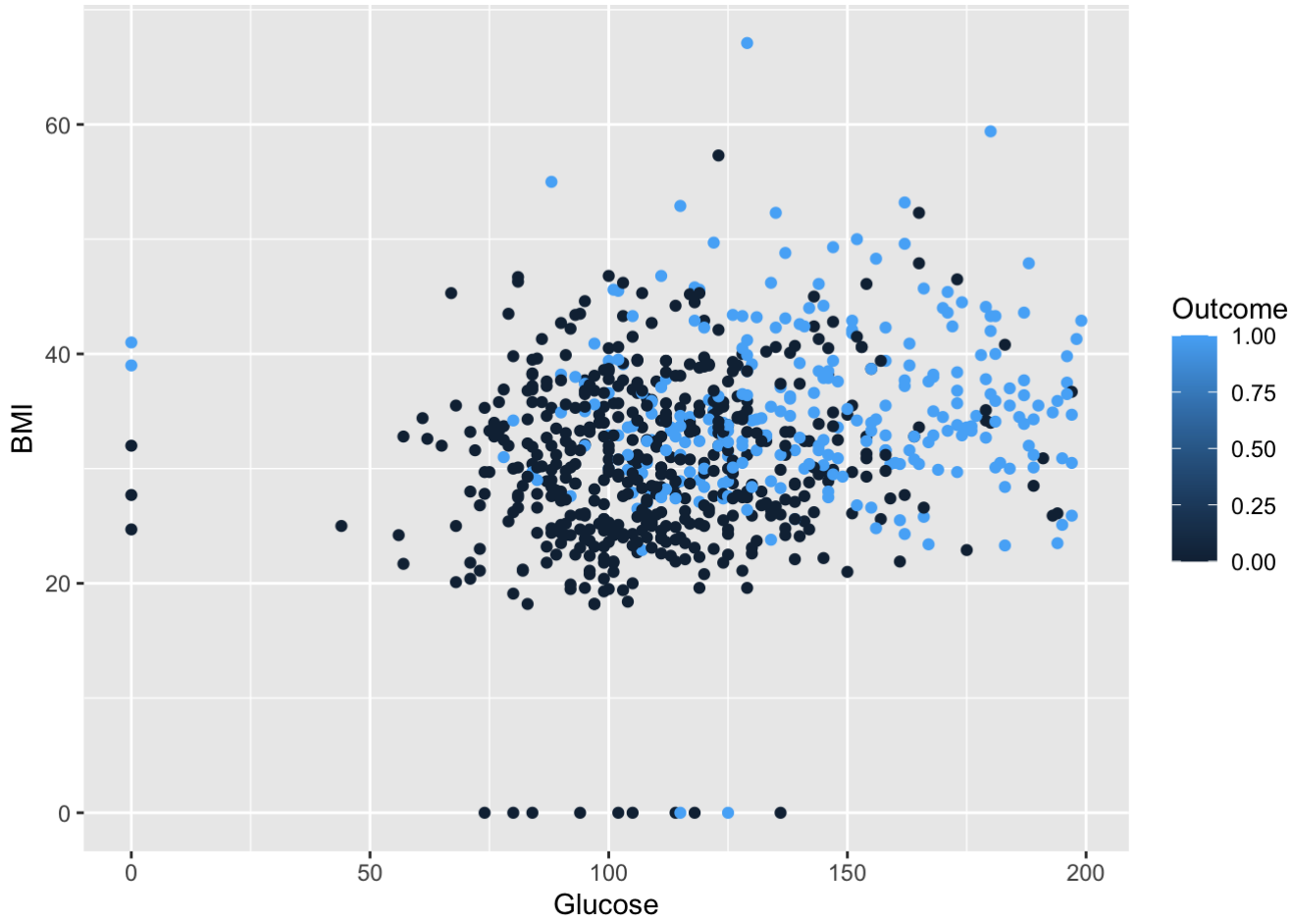


I plotted each of the variables with respect to each other as well as the corresponding diagnosis(Glucose vs. Blood Pressure, Glucose vs. BMI, and Blood Pressure vs. BMI). For the Glucose vs. Blood Pressure plot(the first one), the plot shows that those who were diagnosed with diabetes(light blue) most often had high glucose levels and high blood pressure. For the Glucose vs. BMI plot, the data shows that those who were diagnosed with diabetes(light blue) often had high glucose levels and high BMI. For the Blood Pressure vs. BMI model, there was a less subtle correlation as the data of who was diagnosed with diabetes was scattered.

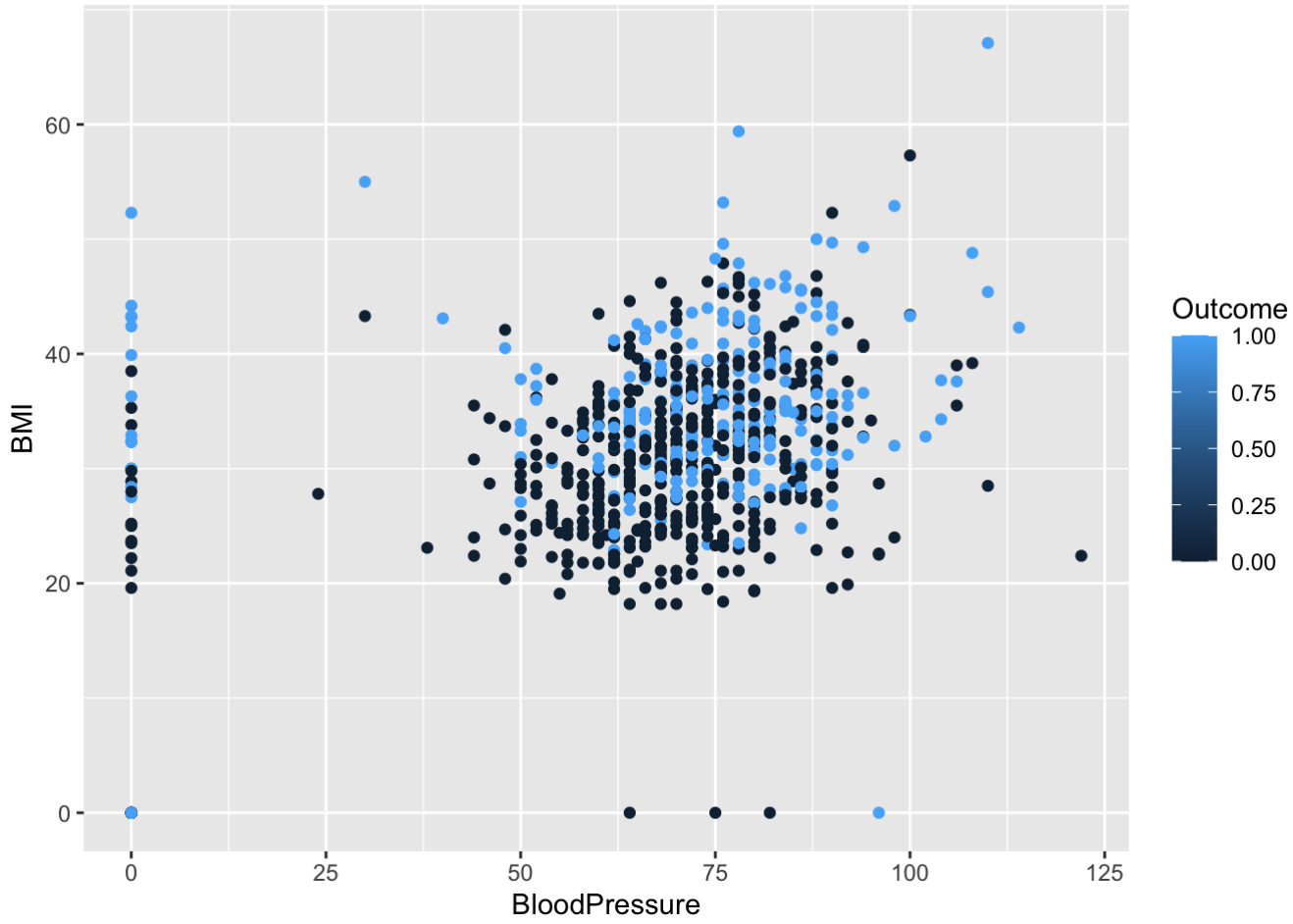
```
library(ggplot2)
ggplot(Diabetes, aes(x =Glucose, y = BloodPressure)) +   geom_point(aes(col = Outcome))
```




```
ggplot(Diabetes, aes(x =Glucose, y = BMI)) +   geom_point(aes(col = Outcome))
```




```
ggplot(Diabetes, aes(x =BloodPressure, y = BMI)) +   geom_point(aes(col = Outcome))
```



From the factors I was examining in this project(Glucose levels, Blood Pressure, and BMI), it seems Glucose has the most correlation to the end Outcome(a positive or negative Diabetes diagnosis) ~ 46.6%, followed by BMI ~ 29.26%, and lastly Blood Pressure which only measured a 6.5% correlation with the Outcome.

```
cor(subset(Diabetes, select = -c(Age)), use ="pairwise.complete.obs")[,"Outcome"]
```

```
##              Pregnancies                  Glucose            BloodPressure
##               0.22189815               0.46658140               0.06506836
##            SkinThickness                  Insulin                      BMI
##               0.07475223               0.13054795               0.29269466
## DiabetesPedigreeFunction                  Outcome
##               0.17384407               1.00000000
```

Splitting the Data into training and testing data

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
set.seed(100)

train_data <- sample(nrow(Diabetes), size = 0.7*nrow(Diabetes))
train_set <- Diabetes[train_data,]
test_set <- Diabetes[-train_data,]
```

Converting each of my quantitative variables(Glucose, Blood Pressure, and BMI) into categorical variables.

```
library(caret)
```

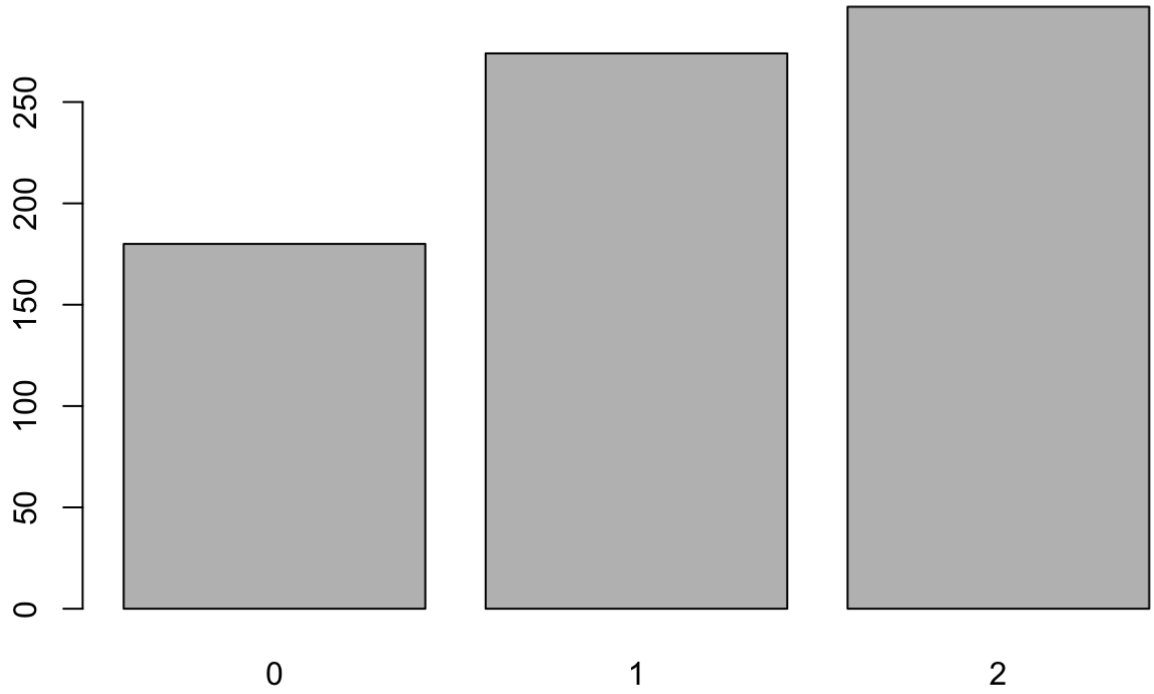
```
## Loading required package: lattice
```

```
Levels1 <- Diabetes %>%
    mutate(glucose_level = factor(ifelse(Glucose < 99, "0",
           ifelse(Glucose >= 100 & Glucose <= 125, "1",
                  ifelse( Glucose>=126 , "2",NA)))))

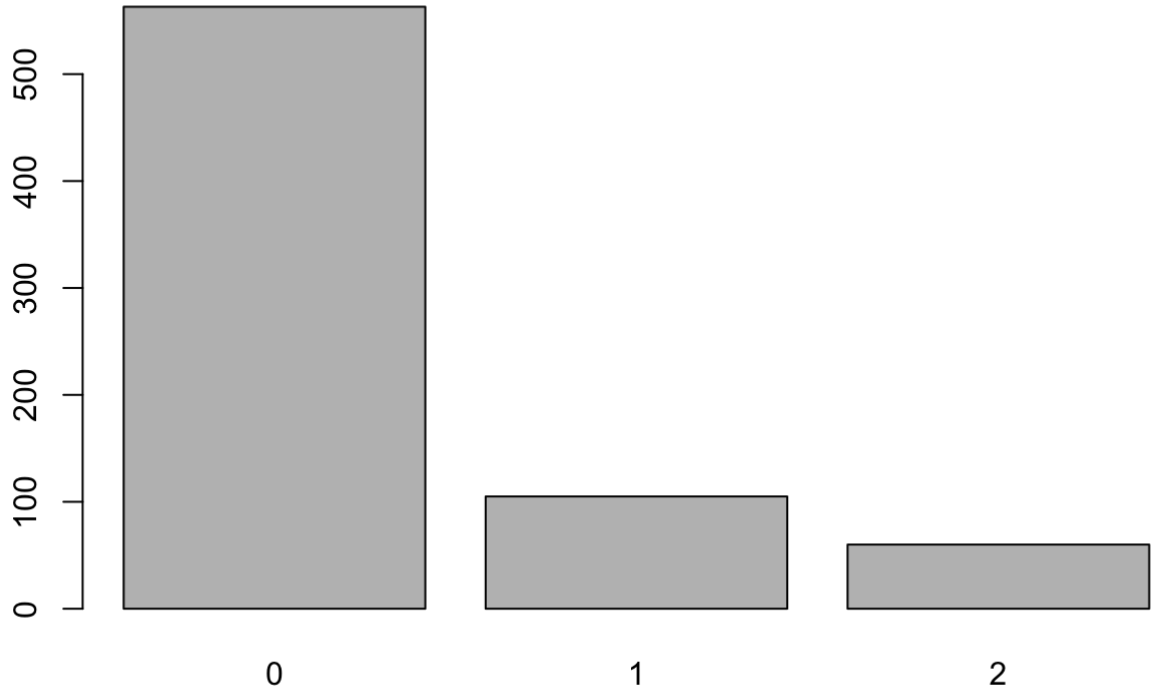
Levels2 <- Diabetes %>%
  mutate(bp_level = factor(ifelse(BloodPressure < 80, "0",
           ifelse(BloodPressure > 80 & BloodPressure <= 89, "1",
                  ifelse(BloodPressure >=90 , "2",NA)))))

Levels3 <- Diabetes %>%
    mutate(bmi_level = factor(ifelse(BMI < 19, "0",
           ifelse(BMI >= 19 & BMI <= 25, "1",
                  ifelse(BMI >25 & BMI <= 30, "2",
                        ifelse(BMI > 30 & BMI <=40, "3",
                               ifelse(BMI > 40  ,"4",NA)))))))

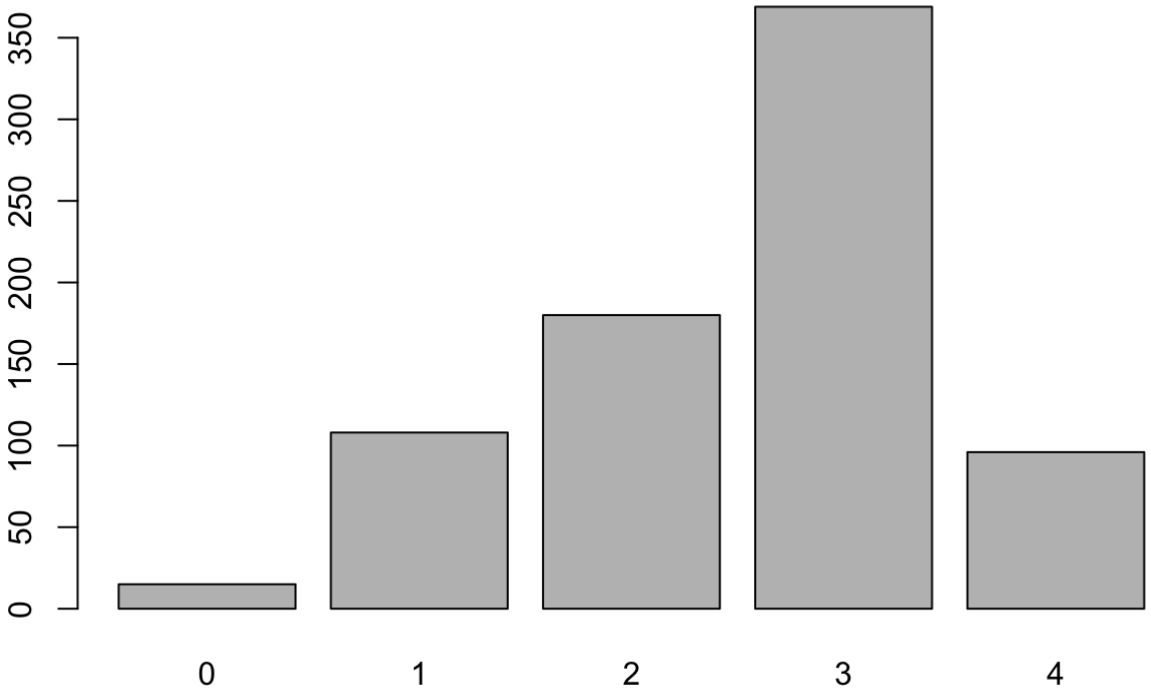
plot(Levels1$glucose_level)
```




```
plot(Levels2$bp_level)
```




```
plot(Levels3$bmi_level)
```



I first tried a SVM Model for my data because it is known to work for both classification and regression, and mine was more classification focused. Also, SVM models can be helpful for both linear and non-linear models.

SVM Model

As seen, the accuracy of testing on the data the model was itself trained on, was a mere 35.13%, which is not desirable at all.

```
library(e1071)
model <- svm(Outcome~.,data = train_set)
predict_svm <- predict(model, train_set)
sqrt(mean((train_set$Outcome -
predict_svm)^2))
```

```
## [1] 0.3513767
```

```
plot(model, train_set)
```

When I tried to predict with the testing data, the accuracy was 39.39%,

```
library(e1071)
model1<- svm(Outcome~.,data = train_set)
predict_svm_test <- predict(model1, test_set)
sqrt(mean((test_set$Outcome -
predict_svm_test)^2))
```

```
## [1] 0.393921
```

The next model I tried was Logistic Regression which is known for being trustworthy in creating classification models.

These were my final results after using Logistic Regression:

Training accuracy:0.7444444 or 74.4% Testing accuracy: 0.7236842 or 72.3% These accuracies are deemed good, and hence this model definitely performed better than svm.

Logistic Regression

Separated data into training and testing sets

```
library(dplyr)
set.seed(100)

set.seed(1)
sample <- sample(c(TRUE, FALSE), nrow(Diabetes), replace=TRUE, prob=c(0.7,0.3))
train_data <- Diabetes[sample, ]
test_data <- Diabetes[!sample, ]
```

Fit a Logistic Regression Curve and created a confusion matrix for testing accuracy. As you can see, the testing accuracy was ~ 72.3% which is pretty good. Because the p value for glucose and BMI is below the threshold value of 0.05, glucose and BMI are considered statistically significant.

```
fit <- glm(Outcome ~ Glucose + BloodPressure + BMI, data = train_data, family = binomia
l)
summary(fit)
```

```
##
## Call:
## glm(formula = Outcome ~ Glucose + BloodPressure + BMI, family = binomial,
##     data = train_data)
##
## Coefficients:
##                Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -7.393530   0.758175  -9.752  < 2e-16 ***
## Glucose        0.038133   0.004058   9.398  < 2e-16 ***
## BloodPressure -0.006861   0.005631  -1.218    0.223
## BMI            0.074868   0.016024   4.672 2.98e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 696.73  on 539  degrees of freedom
## Residual deviance: 530.72  on 536  degrees of freedom
## AIC: 538.72
##
## Number of Fisher Scoring iterations: 4
```

```
predicts <- predict(fit, test_data, type="response")
predicts
```

```
##           4           6           7          15          17          18
## 0.087163635 0.173467250 0.080074831 0.592442718 0.489687237 0.167288088
##          20          21          29          35          37          39
## 0.289436243 0.437753275 0.317594548 0.229672140 0.458332450 0.172474712
##          41          43          46          49          52          61
## 0.828742041 0.092524319 0.896781332 0.270643124 0.111710521 0.014914482
##          68          70          72          76          77          79
## 0.338176501 0.438886707 0.403480082 0.002804790 0.042137157 0.697652897
##          80          82          85          87          94          95
## 0.153967968 0.010234322 0.677694925 0.248747735 0.269747205 0.333571586
##          96          99         104         109         111         112
## 0.535338512 0.114910045 0.056931009 0.113197142 0.755145805 0.654108159
##         117         121         125         135         139         145
## 0.348123837 0.904260817 0.247333515 0.067884928 0.334604606 0.624612361
##         148         150         162         164         165         169
## 0.181367518 0.083327153 0.226771142 0.142363129 0.346126694 0.220374339
##         172         173         176         178         180         183
## 0.471602955 0.128722308 0.800014874 0.857478895 0.482117174 0.002937066
##         185         187         188         189         191         194
## 0.387412840 0.785066441 0.312346776 0.158463685 0.130768086 0.841572890
##         198         200         210         211         213         214
## 0.116679809 0.537889532 0.846101689 0.066445940 0.792635138 0.665620373
##         215         218         219         225         230         243
## 0.245121550 0.299963818 0.076646557 0.093959609 0.475802465 0.366565609
##         250         251         252         260         264         265
## 0.182822605 0.202237525 0.277989731 0.619796018 0.474555311 0.324573652
##         268         280         283         293         300         306
## 0.510762200 0.141069214 0.377552661 0.548364485 0.135896937 0.409314636
##         308         313         314         319         324         325
## 0.314493428 0.500196752 0.228113530 0.352369533 0.448083905 0.275983737
##         326         327         328         330         331         334
## 0.503976439 0.365262635 0.829175291 0.173080751 0.159972307 0.105885092
##         340         341         350         353         355         357
## 0.858753200 0.273390126 0.007593645 0.045019773 0.214183002 0.382983517
##         359         363         366         368         373         378
## 0.129798758 0.218907560 0.191021422 0.082487330 0.124638032 0.154130462
##         384         387         388         390         393         403
## 0.075313369 0.257379615 0.302814856 0.156963819 0.256755512 0.459228474
##         404         405         406         408         410         411
## 0.056395513 0.738230592 0.529813283 0.088837057 0.866819678 0.190199438
##         413         414         415         420         428         429
## 0.658699583 0.380659269 0.511881908 0.281502413 0.835119983 0.537198515
##         430         431         436         437         440         448
## 0.152754143 0.123864714 0.760892130 0.540387819 0.238308680 0.169787170
##         451         458         466         467         472         473
## 0.042347871 0.089507284 0.195102619 0.054828207 0.459085879 0.400423267
##         477         481         483         484         489         492
## 0.195377094 0.691887300 0.078069467 0.130898174 0.100129931 0.108210833
##         499         506         508         509         511         514
## 0.808645221 0.068939925 0.330320951 0.094708225 0.078651635 0.090713474
##         515         521         522         526         527         528
## 0.111820311 0.032012602 0.325555164 0.054395753 0.058901391 0.181110340
```

```
##         529         530         542         547         550         551
## 0.253711669 0.146161316 0.358740907 0.922732639 0.767195345 0.197968003
##         554         555         559         560         564         565
## 0.097554743 0.133904476 0.383783625 0.082682003 0.117520893 0.114393675
##         570         575         579         580         582         589
## 0.339764422 0.431165302 0.317125225 0.903456318 0.144668607 0.772213881
##         590         593         595         601         602         604
## 0.046083024 0.417400067 0.335892269 0.135865283 0.123637523 0.605048892
##         605         606         609         611         615         621
## 0.846964284 0.402159047 0.720519155 0.196462726 0.515940307 0.301998295
##         630         631         634         641         643         645
## 0.082731648 0.192454253 0.265748823 0.130060098 0.467963405 0.130844225
##         648         650         652         659         660         662
## 0.872044883 0.148237406 0.307221442 0.411416156 0.087460501 0.947112490
##         667         671         673         681         682         690
## 0.501573846 0.720583497 0.053657988 0.021237544 0.878250284 0.728370766
##         693         697         699         701         702         703
## 0.417651710 0.686048151 0.360885991 0.360076087 0.250534104 0.736789453
##         709         710         711         714         720         723
## 0.685835048 0.191340957 0.629060996 0.330662185 0.174969578 0.503825719
##         724         726         731         733         735         738
## 0.355550402 0.330031506 0.300438981 0.877545831 0.103422047 0.046836169
##         748         752         759         761         763         768
## 0.206506176 0.402563066 0.256285461 0.090334936 0.060620442 0.113902244
```

```
predicted <- ifelse(predicts>0.7, 1, 0) #above 0.7 would be deemed to be a positive diab
etes diagnosis, and under 0.7 would be a negative diabetes diagnois.
table(predicts, test_data$Outcome) #creating a confusion matrix
```

```
##
## predicts              0 1
##   0.00280479044802127 1 0
##   0.00293706568632395 1 0
##   0.00759364451486289 0 1
##   0.0102343218774618  1 0
##   0.014914481894659   1 0
##   0.0212375435681672  1 0
##   0.0320126022668222  1 0
##   0.0421371567504496  1 0
##   0.0423478708081061  1 0
##   0.0450197725802685  1 0
##   0.0460830244771444  1 0
##   0.0468361687931319  1 0
##   0.053657988471268   1 0
##   0.0543957526461296  1 0
##   0.0548282066478611  1 0
##   0.0563955127366729  1 0
##   0.0569310089298675  1 0
##   0.0589013912659422  1 0
##   0.0606204418080526  1 0
##   0.0664459397120679  1 0
##   0.0678849280908158  1 0
##   0.0689399245545582  1 0
##   0.0753133691122891  1 0
##   0.0766465571243622  0 1
##   0.078069466956657   1 0
##   0.078651634668748   0 1
##   0.0800748311297263  0 1
##   0.0824873296792878  1 0
##   0.0826820026546049  1 0
##   0.0827316480529489  1 0
##   0.083327152734127   1 0
##   0.0871636345390755  1 0
##   0.0874605012820173  0 1
##   0.0888370572491968  1 0
##   0.0895072837196025  1 0
##   0.0903349358099947  1 0
##   0.0907134735738689  1 0
##   0.0925243187591054  1 0
##   0.0939596088151771  1 0
##   0.0947082246541361  1 0
##   0.0975547434805017  1 0
##   0.100129931017799   1 0
##   0.10342204671715    1 0
##   0.105885091567677   1 0
##   0.108210832696129   1 0
##   0.11171052131354    1 0
##   0.111820311234394   1 0
##   0.113197142397347   1 0
##   0.113902243959619   1 0
##   0.114393674739608   1 0
```

```
##   0.114910045210909   1 0
##   0.116679809076541   0 1
##   0.117520893131452   1 0
##   0.123637523438467   1 0
##   0.12386471435821    1 0
##   0.124638031633171   1 0
##   0.128722308068055   1 0
##   0.129798758320697   1 0
##   0.130060098361958   1 0
##   0.130768085711115   1 0
##   0.130844225189549   1 0
##   0.130898174474092   1 0
##   0.133904476466458   1 0
##   0.135865282865949   1 0
##   0.135896937051214   1 0
##   0.141069213819995   1 0
##   0.142363129275795   1 0
##   0.144668607359403   1 0
##   0.146161316424441   1 0
##   0.148237406070957   1 0
##   0.152754143475145   0 1
##   0.153967968125022   1 0
##   0.154130461953449   1 0
##   0.156963819338251   1 0
##   0.158463684796719   0 1
##   0.159972307145458   1 0
##   0.16728808848441    0 1
##   0.169787170073      1 0
##   0.172474712019099   0 1
##   0.173080751416793   1 0
##   0.173467249522526   1 0
##   0.174969578219157   0 1
##   0.181110339961426   1 0
##   0.181367518360183   1 0
##   0.182822605383315   1 0
##   0.190199438200734   1 0
##   0.191021421753225   1 0
##   0.191340957426729   0 1
##   0.192454253498777   0 1
##   0.195102618719391   1 0
##   0.195377093826169   0 1
##   0.196462725861407   1 0
##   0.19796800321234    1 0
##   0.202237524759562   1 0
##   0.206506175688413   1 0
##   0.214183002239944   1 0
##   0.218907559971765   1 0
##   0.220374339104882   1 0
##   0.226771142133597   1 0
##   0.228113529893073   1 0
##   0.229672139548599   1 0
##   0.238308680349713   1 0
```

```
##   0.245121549665506   0 1
##   0.247333514796412   0 1
##   0.248747735051984   1 0
##   0.250534104005688   0 1
##   0.253711669262806   1 0
##   0.256285461393004   1 0
##   0.256755512206348   1 0
##   0.257379615401196   0 1
##   0.265748822856138   1 0
##   0.269747204854149   0 1
##   0.270643124280556   0 1
##   0.273390125586611   1 0
##   0.275983736587965   1 0
##   0.277989730511017   1 0
##   0.281502413236659   0 1
##   0.289436242950985   0 1
##   0.299963817851423   1 0
##   0.300438980603365   0 1
##   0.301998294529124   1 0
##   0.302814855913477   0 1
##   0.307221442289615   1 0
##   0.312346775823465   0 1
##   0.314493427617125   1 0
##   0.3171252247019     1 0
##   0.317594548197424   1 0
##   0.324573652414352   0 1
##   0.325555164392537   1 0
##   0.330031506010312   1 0
##   0.330320950863857   1 0
##   0.330662184694503   1 0
##   0.333571585845512   1 0
##   0.334604606342254   1 0
##   0.335892269126561   1 0
##   0.338176501298857   1 0
##   0.339764422107452   0 1
##   0.346126694380576   0 1
##   0.348123837337818   0 1
##   0.352369532655394   1 0
##   0.355550401903285   1 0
##   0.358740906893032   0 1
##   0.360076086634585   1 0
##   0.360885990784464   1 0
##   0.36526263511609    0 1
##   0.366565609122466   0 1
##   0.377552661124832   1 0
##   0.380659268533227   1 0
##   0.382983516821187   0 1
##   0.383783625268774   1 0
##   0.387412839806467   1 0
##   0.400423267073234   1 0
##   0.402159047482664   1 0
##   0.402563065666909   1 0
```

```
##   0.403480082440486   1 0
##   0.409314635885205   1 0
##   0.411416155542118   1 0
##   0.41740006696607    0 1
##   0.417651710448537   1 0
##   0.431165302488809   1 0
##   0.437753274804785   1 0
##   0.43888670747742    1 0
##   0.448083905481434   0 1
##   0.458332450058154   1 0
##   0.459085878978796   1 0
##   0.459228474393479   0 1
##   0.46796340531171    0 1
##   0.471602954888025   0 1
##   0.474555310850819   1 0
##   0.47580246489141    1 0
##   0.482117174200733   0 1
##   0.489687237484808   0 1
##   0.500196752067278   0 1
##   0.501573846456678   0 1
##   0.503825718743412   0 1
##   0.503976439283303   1 0
##   0.510762200379935   1 0
##   0.511881907701665   0 1
##   0.515940306868528   0 1
##   0.529813283419458   1 0
##   0.535338512034089   1 0
##   0.537198515421409   1 0
##   0.537889531608505   0 1
##   0.54038781865581    1 0
##   0.548364484988797   0 1
##   0.592442717916776   0 1
##   0.605048892020362   0 1
##   0.619796018359368   0 1
##   0.624612361489224   1 0
##   0.629060995533117   1 0
##   0.654108158652544   0 1
##   0.658699582980214   1 0
##   0.665620372859943   0 1
##   0.677694924730372   0 1
##   0.685835047973338   0 1
##   0.686048150858379   0 1
##   0.691887300008106   0 1
##   0.697652896707576   0 1
##   0.720519154904416   1 0
##   0.720583496746989   1 0
##   0.728370765653434   0 1
##   0.736789453130659   0 1
##   0.738230591896599   0 1
##   0.755145804700889   0 1
##   0.760892130010091   0 1
##   0.767195344806525   1 0
```

```
##   0.772213880651175   0 1
##   0.785066440737878   0 1
##   0.792635137764581   1 0
##   0.800014874153195   0 1
##   0.808645221377356   0 1
##   0.828742041166029   1 0
##   0.829175291179529   1 0
##   0.835119983270009   0 1
##   0.841572889843573   0 1
##   0.846101688864028   0 1
##   0.846964284360225   0 1
##   0.857478894607676   0 1
##   0.85875320028011    0 1
##   0.866819677682811   0 1
##   0.872044883057512   0 1
##   0.877545831071999   0 1
##   0.878250283988605   0 1
##   0.896781332281986   0 1
##   0.903456317733816   0 1
##   0.904260817050787   0 1
##   0.922732639475805   0 1
##   0.947112489519374   0 1
```

```
(141+24)/228
```

```
## [1] 0.7236842
```

Finding the training accuracy using a confusion matrix. The training accuracy was ~ 74.4%, which is also considered good. Because the p value for Glucose and BMI is less than the threshold value of 0.05, Glucose and BMI are considered statistically significant.

```
fit <- glm(Outcome ~ Glucose + BloodPressure + BMI, data = train_data, family = binomia
l)
summary(fit)
```

```
##
## Call:
## glm(formula = Outcome ~ Glucose + BloodPressure + BMI, family = binomial,
##     data = train_data)
##
## Coefficients:
##                Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -7.393530   0.758175  -9.752  < 2e-16 ***
## Glucose        0.038133   0.004058   9.398  < 2e-16 ***
## BloodPressure -0.006861   0.005631  -1.218    0.223
## BMI            0.074868   0.016024   4.672 2.98e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 696.73  on 539  degrees of freedom
## Residual deviance: 530.72  on 536  degrees of freedom
## AIC: 538.72
##
## Number of Fisher Scoring iterations: 4
```

```
prediction <- predict(fit, train_data, type="response")
prediction
```

```
##           1           2           3           5           8           9
## 0.567500342 0.068268125 0.708899350 0.686328057 0.409644638 0.872337373
##          10          11          12          13          14          16
## 0.036067377 0.265977708 0.794131122 0.351359991 0.839618069 0.208461357
##          19          22          23          24          25          26
## 0.394147027 0.175895686 0.920026338 0.225575073 0.538587257 0.314570525
##          27          28          30          31          32          33
## 0.654825558 0.082370688 0.266933233 0.257996285 0.616756562 0.070497110
##          34          36          38          40          42          44
## 0.046234910 0.110986194 0.173318307 0.293757240 0.527838347 0.854635501
##          45          47          48          50          51          53
## 0.569993214 0.503296647 0.044358149 0.032621689 0.071602866 0.065136540
##          54          55          56          57          58          59
## 0.772653804 0.615726194 0.038017849 0.890243820 0.336212407 0.655515899
##          60          62          63          64          65          66
## 0.327020795 0.412719046 0.013797404 0.374459572 0.260460872 0.124014427
##          67          69          71          73          74          75
## 0.196606035 0.059728747 0.172224461 0.510736482 0.392548475 0.075868075
##          78          81          83          84          86          88
## 0.191173181 0.153242562 0.071090564 0.104672153 0.217390027 0.237868149
##          89          90          91          92          93          97
## 0.522442589 0.142351474 0.035905300 0.298106158 0.206916572 0.125111843
##          98         100         101         102         103         105
## 0.029651557 0.589546955 0.776942549 0.476736530 0.167828611 0.163357934
##         106         107         108         110         113         114
## 0.304844759 0.052509837 0.477072514 0.174500554 0.101079324 0.085073764
##         115         116         118         119         120         122
## 0.650368514 0.469659204 0.097487012 0.119723068 0.082951035 0.261259179
##         123         124         126         127         128         129
## 0.213241335 0.288546275 0.468556034 0.478499833 0.310299196 0.278312778
##         130         131         132         133         134         136
## 0.132719659 0.720452922 0.346951048 0.774316419 0.138159137 0.375643440
##         137         138         140         141         142         143
## 0.147466492 0.108114079 0.245830610 0.187244926 0.277520215 0.231693369
##         144         146         147         149         151         152
## 0.213775299 0.017660666 0.035121590 0.549796162 0.521198928 0.135547033
##         153         154         155         156         157         158
## 0.630199276 0.714591445 0.944086501 0.823791878 0.105890106 0.149996944
##         159         160         161         163         166         167
## 0.085145011 0.800622075 0.492643053 0.428951132 0.154869598 0.557364208
##         168         170         171         174         175         177
## 0.255768341 0.160830636 0.146705151 0.177123976 0.060136738 0.086943307
##         179         181         182         184         186         190
## 0.709576706 0.052752682 0.335822657 0.046750484 0.902507264 0.431395683
##         192         193         195         196         197         199
## 0.330623648 0.620743724 0.062806000 0.732018476 0.122576007 0.255255100
##         201         202         203         204         205         206
## 0.212064405 0.576486589 0.154743923 0.071003570 0.242808209 0.134067064
##         207         208         209         212         216         217
## 0.914241850 0.709492091 0.156264263 0.697014413 0.733743642 0.272457921
##         220         221         222         223         224         226
## 0.321803059 0.822693812 0.593812265 0.275064995 0.441723669 0.117044120
```

```
##         227         228         229         231         232         233
## 0.199281490 0.770699415 0.915751082 0.673803584 0.651640004 0.046163816
##         234         235         236         237         238         239
## 0.357202673 0.056535464 0.869596040 0.834780948 0.892485586 0.643322776
##         240         241         242         244         245         246
## 0.070987682 0.101887430 0.127237583 0.236876180 0.625355898 0.783420261
##         247         248         249         253         254         255
## 0.294832948 0.899930275 0.378707927 0.063941705 0.130062843 0.095836958
##         256         257         258         259         261         262
## 0.267353786 0.215598167 0.203549294 0.826652170 0.850262269 0.557052808
##         263         266         267         269         270         271
## 0.136114306 0.151234024 0.642557624 0.121148907 0.557914247 0.327788109
##         272         273         274         275         276         277
## 0.140164480 0.174428964 0.060901605 0.219040968 0.263396607 0.144409548
##         278         279         281         282         284         285
## 0.143622940 0.155785881 0.629728177 0.423577090 0.606291009 0.141527281
##         286         287         288         289         290         291
## 0.316770688 0.698061159 0.492045587 0.071772942 0.256074725 0.094461996
##         292         294         295         296         297         298
## 0.269225014 0.547424060 0.510590659 0.644951082 0.447656083 0.295939690
##         299         301         302         303         304         305
## 0.201770622 0.801047304 0.516356460 0.087891776 0.569510523 0.349131106
##         307         309         310         311         312         315
## 0.546959713 0.332779970 0.338836259 0.055503170 0.292774148 0.250076994
##         316         317         318         320         321         322
## 0.261877468 0.061667836 0.789572805 0.773541907 0.304232885 0.220188037
##         323         329         332         333         335         336
## 0.250869053 0.334575972 0.116525642 0.937738491 0.083688548 0.876866504
##         337         338         339         342         343         344
## 0.400958723 0.232572336 0.605382620 0.087898082 0.004217434 0.324429882
##         345         346         347         348         349         351
## 0.180970421 0.423070808 0.435364250 0.229568657 0.082287134 0.218439320
##         352         354         356         358         360         361
## 0.398956209 0.087020179 0.638891442 0.625466236 0.908186940 0.846875139
##         362         364         365         367         369         370
## 0.594405525 0.627400878 0.578625574 0.251104297 0.055410883 0.362146947
##         371         372         374         375         376         377
## 0.819908831 0.034456172 0.236014655 0.404221589 0.578654031 0.088469015
##         379         380         381         382         383         385
## 0.839794684 0.216864470 0.182215916 0.086368486 0.148413853 0.216201356
##         386         389         391         392         394         395
## 0.174121350 0.482689095 0.162829393 0.862535177 0.140690397 0.636316133
##         396         397         398         399         400         401
## 0.294258450 0.093826597 0.429713853 0.040418089 0.890774393 0.140123413
##         402         407         409         412         416         417
## 0.315163669 0.207741130 0.824905964 0.260915189 0.785811945 0.106708431
##         418         419         421         422         423         424
## 0.602854427 0.034473470 0.483005759 0.088745257 0.288337972 0.242053075
##         425         426         427         432         433         434
## 0.739074924 0.865042292 0.021687984 0.096960594 0.068842842 0.333796587
##         435         438         439         441         442         443
## 0.069532997 0.483988216 0.056660433 0.841301648 0.093586916 0.292037843
```

```
##         444         445         446         447         449         450
## 0.186637047 0.243456069 0.967147952 0.101550920 0.205660688 0.260790167
##         452         453         454         455         456         457
## 0.354262546 0.197375287 0.199673338 0.245871331 0.797355955 0.350429021
##         459         460         461         462         463         464
## 0.619823521 0.298928734 0.147497338 0.029904627 0.082477596 0.075391905
##         465         468         469         470         471         474
## 0.131948375 0.201228761 0.360873703 0.801411915 0.651809861 0.357451072
##         475         476         478         479         480         482
## 0.210512540 0.331412682 0.143563695 0.239128261 0.298640546 0.338129015
##         485         486         487         488         490         491
## 0.809207667 0.611764626 0.629166707 0.895632147 0.803664235 0.127934172
##         493         494         495         496         497         498
## 0.163928319 0.280184292 0.012831648 0.603541885 0.152008816 0.072745561
##         500         501         502         503         504         505
## 0.541106748 0.159380584 0.130188027 0.007102196 0.147408985 0.186125221
##         507         510         512         513         516         517
## 0.829984676 0.185263416 0.296523635 0.070553816 0.669878084 0.450269495
##         518         519         520         523         524         525
## 0.400838708 0.079332984 0.164582980 0.045372314 0.411908154 0.340948727
##         531         532         533         534         535         536
## 0.284560288 0.390964493 0.186198210 0.155461578 0.087190400 0.525760847
##         537         538         539         540         541         543
## 0.142948451 0.017862963 0.405676320 0.406018253 0.242653108 0.126541063
##         544         545         546         548         549         552
## 0.135802347 0.101812185 0.840851493 0.404540135 0.679891741 0.093752440
##         553         556         557         558         561         562
## 0.172378722 0.225092449 0.210407585 0.162593281 0.350268368 0.942456055
##         563         566         567         568         569         571
## 0.150906652 0.100889790 0.227497955 0.128426813 0.581335447 0.078258206
##         572         573         574         576         577         578
## 0.197349112 0.205846109 0.186900282 0.377582550 0.144270335 0.442522600
##         581         583         584         585         586         587
## 0.710731092 0.209051683 0.230707842 0.261570687 0.072635676 0.554653970
##         588         591         592         594         596         597
## 0.109153848 0.441934068 0.330031506 0.076583856 0.833211481 0.122543450
##         598         599         600         603         607         608
## 0.110754115 0.810126120 0.145747936 0.251339687 0.877398745 0.054640247
##         610         612         613         614         616         617
## 0.143150118 0.786969557 0.780558938 0.181638723 0.128545556 0.191240181
##         618         619         620         622         623         624
## 0.023636220 0.171645126 0.394120938 0.069462853 0.880208439 0.262584417
##         625         626         627         628         629         632
## 0.196489038 0.148951750 0.223693201 0.384741714 0.384376720 0.189045705
##         633         635         636         637         638         639
## 0.166458671 0.085361207 0.169967682 0.144394207 0.122961551 0.239758136
##         640         642         644         646         647         649
## 0.067353691 0.395354142 0.134087979 0.737955032 0.554482552 0.339602334
##         651         653         654         655         656         657
## 0.082618592 0.341202052 0.234754693 0.219040968 0.742239164 0.090465854
##         658         661         663         664         665         666
## 0.388377214 0.569903738 0.743143603 0.604513350 0.289689294 0.256134181
```

```
##         668         669         670         672         674         675
## 0.170481448 0.181092372 0.563934009 0.107676328 0.711075481 0.139341458
##         676         677         678         679         680         683
## 0.867089638 0.455573134 0.165754454 0.391499903 0.106378762 0.295062006
##         684         685         686         687         688         689
## 0.319166056 0.058960447 0.378356886 0.241228827 0.176879874 0.318999815
##         691         692         694         695         696         698
## 0.117069475 0.734226103 0.485364398 0.068251453 0.420623347 0.148464006
##         700         704         705         706         707         708
## 0.489369228 0.600608154 0.168802645 0.128749699 0.047052906 0.427798507
##         712         713         715         716         717         718
## 0.287430462 0.546057147 0.141471705 0.873496620 0.768307724 0.070855919
##         719         721         722         725         727         728
## 0.263266697 0.067550098 0.343717242 0.205856530 0.309474358 0.458236548
##         729         730         732         734         736         737
## 0.596371505 0.120407291 0.217308075 0.172998583 0.177662042 0.244623472
##         739         740         741         742         743         744
## 0.215870388 0.258374066 0.450268395 0.182432044 0.182234419 0.437398327
##         745         746         747         749         750         751
## 0.706121797 0.128917678 0.778684139 0.878913665 0.544346069 0.412923583
##         753         754         755         756         757         758
## 0.147539635 0.895339219 0.591325142 0.405246491 0.403455311 0.382367980
##         760         762         764         765         766         767
## 0.867410391 0.867091574 0.167922581 0.385448424 0.212161614 0.321483735
```

```
prediction <- ifelse(prediction>0.7, 1, 0) #above 0.7 would be deemed to be a positive d
iabetes diagnosis, and under 0.7 would be a negative diabetes diagnois.
table(prediction, train_data$Outcome) #creating a confusion matrix
```

```
##
## prediction   0   1
##          0 339 124
##          1  14  63
```

```
(339+63)/540
```

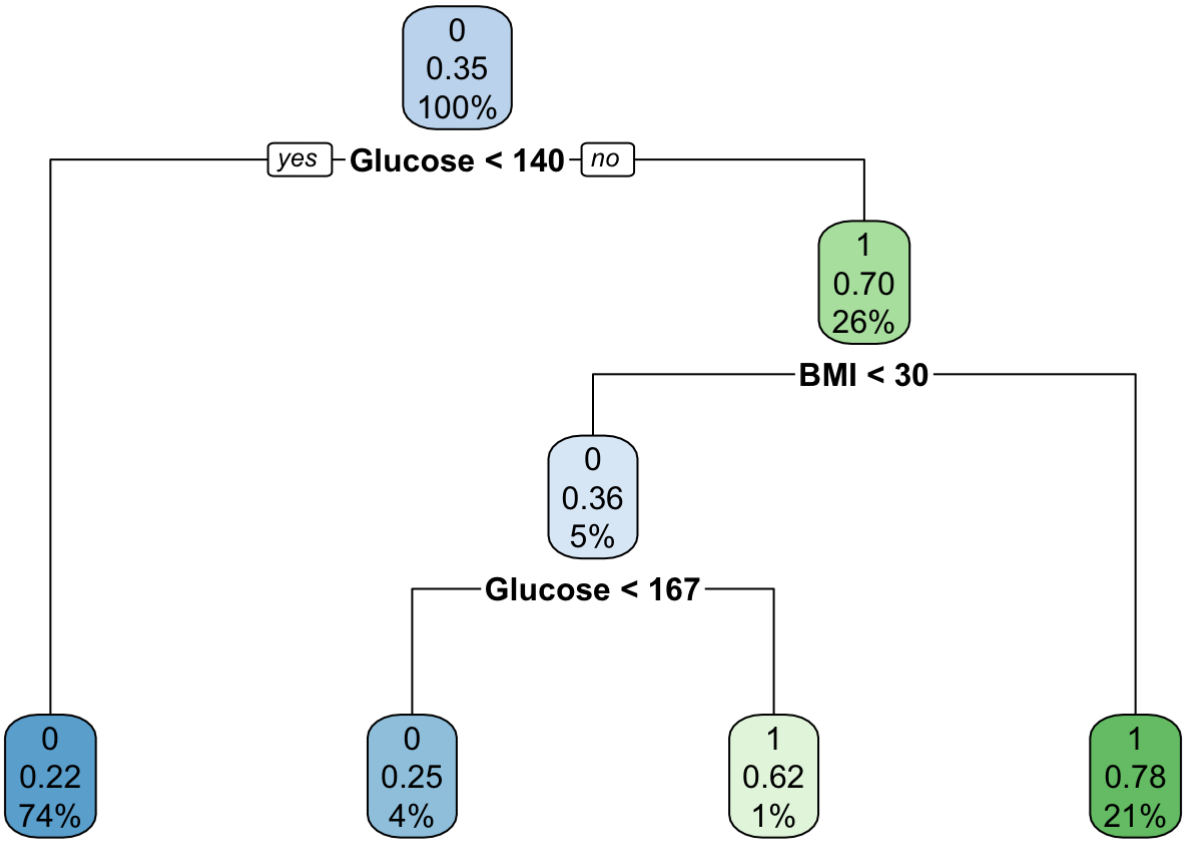
```
## [1] 0.7444444
```

I then tried to see if I could futher better the accuracy. I decided to use a decision tree model which are known to be good for classification models.

The Training accuracy is approximately 77.5% which is slightly better than the logistic regression model, and the testing accuracy was ~ 72.3% which is very similar to the logistic regression model. So the results were quite similar.

Creating a Decision Tree

```
library(rpart)
tree1 <- rpart(formula = Outcome ~ BMI + Glucose + BloodPressure,
data = train_data, method = "class")
library(rpart.plot)
rpart.plot(tree1)
```



Predicting with the Decision tree using a confusion matrix

```
my_prediction <- predict(tree1, test_data,
type ="class")
table(my_prediction, test_data$Outcome)
```

```
##
## my_prediction   0   1
##             0 134  50
##             1  13  31
```

Testing accuracy was 72.3%

```
(134+31)/228
```

```
## [1] 0.7236842
```

Testing for Training accuracy

```
my_prediction1 <- predict(tree1, train_data,
type ="class")
table(my_prediction1, train_data$Outcome)
```

```
##
## my_prediction1   0   1
##              0 325  93
##              1  28  94
```

Training accuracy was 77.5%

```
(325+94)/540
```

```
## [1] 0.7759259
```

Analysis

Overall, one cannot find out solely from these values if someone will be diagnosed with diabetes. There are several factors that influence every single diagnosis, and each case differs from others significantly. For some positively diagnosed patients, their Blood Pressure was high, whereas, for others, their blood pressure was low. Also, after fitting the Logistic Regression Model, by analyzing the variable -p values, because the p values for Glucose and BMI were less than the threshold value of 0.05, Glucose and BMI are the statistically significant predictors. This is only reinforced by the fact that when initially observing the correlations between these three predictors, the Blood Pressure variable was the only one out of the three where the diagnosis of the patients varied widely. After testing with three different models, the models which gave me the best, yet similar accuracies were the Logistic Regression Model and Decision Tree Model. For the Logistic Regression Model, my training and testing accuracies were 74.4% and 72.3% respectively. For the Decision Tree Model, my training and testing accuracies were 77.5% and 72.3% respectively. Hence, both of these models could only accurately predict ~ 73-74% of the data, which is good, but not good enough if we want to actually apply such models in the Healthcare setting because it can result in False Positives and False Negatives.